

Intermediate (.env by hand) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real config parser handles the messy edge cases). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A `.env` file is just lines of text like `KEY=value`. To read it yourself, open the file, look at each line one at a time, and split each line into the part before the `=` and the part after.

Skip any line that's empty.

Things to use:

- `open(path)` then a `for` loop to go through each line.
- `.strip()` to clean up each line.
- `.split("=", 1)` to break it into key and value.
- A dictionary (`{}`) to store the results.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Parsing this by hand touches a few real Python skills: reading a file line by line, string splitting with a limit, and building a dictionary as you go.

`for line in open(path):` iterates over a file one line at a time without loading the whole thing into memory first. `line.strip()` removes the trailing newline character each line comes with. `line.split("=", 1)` splits on the first `=` only (the second argument, `1`, is the max number of splits) — this matters because a value could itself contain an `=` character, and you don't want to split on that one too.

The exact pieces:

- `open(path)` used in a `for line in open(path):` loop — Python handles closing the file for you at the end of iteration in this pattern (though a `with open(path) as f:` block is the more correct, explicit way — worth knowing for later).
- `line.strip()` — removes leading/trailing whitespace, including the newline `\n` at the end of each line.
- Skip logic: `if not line or "=" not in line: continue` — handles blank lines and any malformed lines safely.
- `key, value = line.split("=", 1)` — unpacks the 2-item list `split()` returns directly into two variables.
- For the library comparison: `from dotenv import load_dotenv; load_dotenv(); os.getenv("KEY")`.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about what a real `.env` file looks like, not just the tidy 2-line one in this exercise. Real ones have comment lines (`# this is a note`), values wrapped in quotes (`API_KEY="abc123"`), and sometimes a required key that's just missing entirely — someone forgot to set it before deploying.

A parser that only handles the tidy case will either crash on a comment line, keep the quote marks as part of the value by mistake, or return a dictionary that's silently missing a key your program actually needs — and that last one is the dangerous one, because the program won't fail until much later, somewhere confusing, when it tries to *use* the missing value.

The real design question isn't just "split each line on `=`" — it's "what should this function do when the input isn't perfectly clean, and how do I make sure a genuinely missing required key gets caught immediately instead of causing a confusing failure later?"

The extra pieces needed to handle comments, quotes, and a required-key check:

- `line.startswith("#")` — added to your skip condition, so comment lines get skipped just like blank lines.
- `value.strip().strip('"').strip('\'')` — after splitting, this trims whitespace around the value, then strips a leading/trailing double quote, then a single quote, so `API_KEY="abc123"` becomes `abc123`, not `"abc123"`.
- A second loop, after the parsing loop finishes, that checks a list of required key names against the dictionary you just built, and collects any that are missing.
- A custom exception (like `MissingEnvKeyError(Exception)`) raised with all the missing key names in the message, not just the first one — so whoever sees the error learns everything that's wrong in one shot, instead of fixing one key, rerunning, and discovering the next missing one.

Sketch this required-key check yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic describes the plain idea and lists the tools for the tidy, happy-path case. Intermediate shows the real Python syntax for that same tidy case. Advanced adds the pieces that only matter once the file isn't perfectly tidy — comments, quoted values, and a check that nothing required is silently missing — which is the difference between a parser that works on the example file and one that would survive contact with a real project's `.env`.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make an empty dictionary called env_vars

open the file, go through it line by line:
    remove extra spaces from the line
    if the line is empty, skip to the next one
    if there's no "=" in the line, skip it too
    split the line at the first "=" into key and value
    save env_vars[key] = value

return env_vars
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# env_config_practice.py – Intermediate section
def parse_env_by_hand(path):
    env_vars = {}
    for line in open(path):
        line = line.strip()
        if line == "" or "=" not in line:
            continue
        parts = line.split("=", 1)
        key = parts[0]
        value = parts[1]
        env_vars[key] = value
    return env_vars
```

Expected output if you run just this (nothing calls the function yet): nothing — defining a function doesn't run it. Add a call like `print(parse_env_by_hand(".env"))` below it, against a `.env` file containing `OPENAI_API_KEY=sk-test123`, to see it print `{'OPENAI_API_KEY': 'sk-test123'}`.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
function parse_env_by_hand(path) -> dict:
    env_vars = {}
    for line in open(path):
        line = line.strip()
        if not line or "=" not in line:
            continue
        key, value = line.split("=", 1)
        env_vars[key] = value
    return env_vars
```

```
compare against:
from dotenv import load_dotenv
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")
```

```
# env_config_practice.py – Intermediate section
def parse_env_by_hand(path: str) -> dict:
    env_vars = {}
    for line in open(path):
        line = line.strip()
        if not line or "=" not in line:
            continue
        key, value = line.split("=", 1)
        env_vars[key] = value
    return env_vars
```

Now write the 2-line `python-dotenv` version yourself and compare both against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

make a `MissingEnvKeyError`, it's a kind of `Exception`

```
function parse_env_by_hand(path, required_keys) -> dict:
    env_vars = {}
    for line in open(path):
        line = line.strip()
        if line is empty, or starts with "#", or has no "=": skip it
        split line at first "=" into key and value
        clean value: strip spaces, then strip a leading/trailing quote
        env_vars[key] = value

    missing_keys = empty list
    for each name in required_keys:
        if name is not in env_vars: add it to missing_keys

    if missing_keys is not empty:
        raise MissingEnvKeyError, naming everything in missing_keys

    return env_vars
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# env_config_practice.py – Intermediate section
class MissingEnvKeyError(Exception):
    pass

def parse_env_by_hand(path: str, required_keys: list[str]) -> dict[str, str]:
    env_vars: dict[str, str] = {}
    for line in open(path):
        line = line.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        key, value = line.split("=", 1)
        value = value.strip().strip('\'').strip('"')
        env_vars[key] = value

    # your turn: check required_keys against env_vars, and raise
    # MissingEnvKeyError naming every key that's missing, not just the first one
```

```
...  
  
return env_vars
```

Fill in the required-key check yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same underlying idea (open, split, collect) at 3 completeness levels, shown here both as pseudocode and as near-complete code — Basic's version only ever collects whatever key/value pairs it finds, Intermediate adds the type contract real Python expects while still just collecting and returning no matter what, and Advanced adds a second pass *after* parsing that checks the result actually contains what the caller needs — raising loudly, naming every missing key, if it doesn't. That check is the entire point of Hint 1's Advanced question: a parser that "works" but silently returns an incomplete dictionary is more dangerous than one that fails immediately and clearly.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)